

exercise 1 : MyArray class

```
#include <iostream>
```

```
using namespace std;
```

```
class MyArray {
```

```
    int *arr;
```

```
    int size;
```

```
    int capacity;
```

```
public:
```

```
    MyArray(int cap) {
```

```
        capacity = cap;
```

```
        size = 0;
```

```
        arr = new int[capacity];
```

```
    }
```

```
    ~MyArray() {
```

```
        delete[] arr;
```

```
    }
```

```
    void insertAt(int index, int value) {
```

```
        if (size == capacity || index < 0 || index > size) return;
```

```
        for (int i = size; i > index; i--)
```

```
            arr[i] = arr[i - 1];
```

```
        arr[index] = value;
```

```
        size++;
```

```
    }
```

```
    void remove(int index) {
```

```
        if (index < 0 || index >= size) return;
```

```
        for (int i = index; i < size - 1; i++)
```

```
            arr[i] = arr[i + 1];
```

```
        size--;
```

```
    }
```

```
    void reverse() {
```

```
        for (int i = 0; i < size / 2; i++)
```

```
            swap(arr[i], arr[size - i - 1]);
```

```
    }
```

```
    void sort() {
```

```
        for (int i = 0; i < size - 1; i++) {
```

```
            int minIdx = i;
```

```

        for (int j = i + 1; j < size; j++) {
            if (arr[j] < arr[minIdx])
                minIdx = j;
        }
        swap(arr[i], arr[minIdx]);
    }
}

void print() {
    for (int i = 0; i < size; i++)
        cout << arr[i] << " ";
    cout << endl;
}

};

int main() {
    MyArray a(10);

    a.insertAt(0, 5);
    a.insertAt(1, 2);
    a.insertAt(2, 8);
    a.insertAt(1, 6);

    a.print();

    a.remove(2);
    a.print();

    a.reverse();
    a.print();

    a.sort();
    a.print();

    return 0;
}

```

exercise 2 : Point2D class

```

#include <iostream>
#include <cmath>
using namespace std;

```

```

class Point2D {
    double x, y;

public:
    Point2D(double xVal = 0, double yVal = 0) {
        x = xVal;
        y = yVal;
    }

    void set(double xVal, double yVal) {
        x = xVal;
        y = yVal;
    }

    double distanceFromOrigin() {
        return sqrt(x * x + y * y);
    }

    double distanceFrom(const Point2D& other) {
        return sqrt((x - other.x) * (x - other.x) +
                    (y - other.y) * (y - other.y));
    }

    void translate(double dx, double dy) {
        x += dx;
        y += dy;
    }

    void print() {
        cout << "(" << x << ", " << y << ")" << endl;
    }
};

int main() {
    Point2D p1(3, 4);
    Point2D p2(6, 8);

    p1.print();
    p2.print();

    cout << "Distance from origin: " << p1.distanceFromOrigin() << endl;
    cout << "Distance between p1 and p2: " << p1.distanceFrom(p2) << endl;

    p1.translate(1, 1);
    p1.print();

    return 0;
}

```

exercise 3 : polynomial class

```
#include <iostream>
using namespace std;

class Polynomial {
    int degree;
    double* coeff;

public:
    Polynomial(int deg) {
        degree = deg;
        coeff = new double[degree + 1];
        for (int i = 0; i <= degree; i++)
            coeff[i] = 0;
    }

    ~Polynomial() {
        delete[] coeff;
    }

    void setCoefficient(int power, double value) {
        if (power >= 0 && power <= degree)
            coeff[power] = value;
    }

    double evaluate(double x) {
        double result = 0;
        for (int i = 0; i <= degree; i++)
            result += coeff[i] * pow(x, i);
        return result;
    }

    Polynomial add(const Polynomial& other) {
        int maxDeg = max(degree, other.degree);
        Polynomial result(maxDeg);

        for (int i = 0; i <= maxDeg; i++) {
            double val = 0;
            if (i <= degree) val += coeff[i];
            if (i <= other.degree) val += other.coeff[i];
            result.coeff[i] = val;
        }
    }
}
```

```

        return result;
    }

    void print() {
        for (int i = degree; i >= 0; i--) {
            cout << coeff[i] << "x^" << i;
            if (i != 0) cout << " + ";
        }
        cout << endl;
    }
};

int main() {
    Polynomial p1(2);
    p1.setCoefficient(0, 1);
    p1.setCoefficient(1, 2);
    p1.setCoefficient(2, 3);

    Polynomial p2(2);
    p2.setCoefficient(0, 2);
    p2.setCoefficient(1, 1);
    p2.setCoefficient(2, 4);

    p1.print();
    p2.print();

    Polynomial p3 = p1.add(p2);
    p3.print();

    cout << "Value at x=2: " << p1.evaluate(2) << endl;

    return 0;
}

```

exercise 4 : Matrix using 1D array

```

#include <iostream>
using namespace std;

class Matrix {
    int rows, cols;
    float* data;

public:
    Matrix(int r, int c) {

```

```

        rows = r;
        cols = c;
        data = new float[r * c];
    }

    ~Matrix() {
        delete[] data;
    }

    void set(int i, int j, float value) {
        data[i * cols + j] = value;
    }

    float get(int i, int j) const {
        return data[i * cols + j];
    }

    void print() {
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++)
                cout << get(i, j) << " ";
            cout << endl;
        }
    }
};

int main() {
    Matrix m(2, 3);

    m.set(0, 0, 1);
    m.set(0, 1, 2);
    m.set(0, 2, 3);
    m.set(1, 0, 4);
    m.set(1, 1, 5);
    m.set(1, 2, 6);

    m.print();

    return 0;
}

```